

Tối ưu, rồi tối ưu nữa!

Phân tích việc tối ưu hóa quy hoạch động qua bài *Trứng đại bàng*

Zhu Chenguang

Nguồn gốc: Optimize, and optimize again!

IOI China candidate team papers / 2004 / Zhu Chenguang.pdf

Từ khóa

Tối ưu hóa; quy hoạch động; mô hình.

Tóm tắt

Bài viết này giới thiệu năm thuật toán có hiệu năng khác nhau cho bài *Trứng đại bàng* của Ural 1223. Từ đó, bài viết tổng kết tư tưởng bản chất và phương pháp chung khi tối ưu hóa thuật toán quy hoạch động.

Toàn văn gồm bốn phần. Phần mở đầu nêu tầm quan trọng của việc tối ưu hóa quy hoạch động. Phần thứ hai trình bày bài toán được thảo luận. Phần thứ ba phân tích chi tiết năm thuật toán quy hoạch động khác nhau cho bài toán này, đồng thời giải thích tư tưởng tối ưu hóa trong từng bước. Phần cuối tổng kết toàn bài, nhấn mạnh lại tầm quan trọng của việc tối ưu hóa quy hoạch động, rồi phân tích tư tưởng bản chất và phương pháp chung của công việc ấy.

1 Mở đầu

Trong các kỳ thi tin học hiện nay, quy hoạch động là một dạng thuật toán rất thường dùng. Nó được ưa chuộng vì tính hiệu quả. Tuy nhiên, quy hoạch động đôi khi vẫn gặp vấn đề độ phức tạp thời gian quá cao. Vì vậy, muốn thật sự dùng tốt và linh hoạt quy hoạch động thì cũng phải nắm được các phương pháp tối ưu hóa nó.

Có nhiều cách tối ưu hóa quy hoạch động, chẳng hạn bất đẳng thức tứ giác, tối ưu bằng độ dốc, v.v. Nhưng các phương pháp ấy chỉ tối ưu được một số mô hình quy hoạch động đặc thù, chưa có ý nghĩa phổ quát. Bài viết này phân tích tương đối sâu bài *Trứng đại bàng*, rồi từ đó bàn về tư tưởng bản chất và phương pháp chung khi tối ưu hóa quy hoạch động.

2 Bài toán

Có một chồng gồm M quả trứng đại bàng. Một giáo sư muốn nghiên cứu độ bền E của những quả trứng này bằng cách liên tục thả trứng từ một tòa nhà N tầng xuống. Khi trứng được thả từ tầng E hoặc thấp hơn thì nó không vỡ; khi được thả từ tầng $E + 1$ hoặc cao hơn thì nó sẽ vỡ. Nếu một quả trứng không vỡ, ta có thể dùng lại nó. Nếu toàn bộ trứng đã vỡ mà vẫn chưa xác định được E , thí nghiệm xem như thất bại. Giáo sư muốn thí nghiệm chắc chắn thành công.

Ví dụ, nếu trứng thả từ tầng 1 đã vỡ thì $E = 0$; nếu thả từ tầng N vẫn không vỡ thì $E = N$.

Giả sử mọi quả trứng đều có cùng độ bền. Cho số trứng M và số tầng N . Hãy tìm số lần thả ít nhất cần dùng trong trường hợp xấu nhất để xác định E .

Ví dụ. Với $M = 1, N = 10$, đáp án là 10. Để không làm thí nghiệm thất bại, ta chỉ có thể thả quả trứng duy nhất lần lượt từ tầng 1 đến tầng 10. Khi nó vỡ ở tầng $E + 1$ thì E được xác định. Ở đây có thể xem rằng nếu thả từ tầng $N + 1$ thì trứng sẽ vỡ.

3 Phân tích

3.1 Thuật toán 1

Nhìn thoáng qua, thuật toán cho bài này không thật rõ ràng, vì đây không phải là tìm kiếm nhị phân đơn giản: số quả trứng cũng bị giới hạn. Nhưng vì đề yêu cầu giá trị tối ưu, ta tự nhiên nghĩ đến quy hoạch động.

Định nghĩa trạng thái: $f(i, j)$ là số lần thả ít nhất cần dùng trong trường hợp xấu nhất để xác định E khi có i quả trứng và một tòa nhà j tầng.

Rõ ràng, khi số tầng bằng 0 thì không cần thả:

$$f(i, 0) = 0 \quad (i \geq 0).$$

Khi chỉ có một quả trứng, để không thất bại ta chỉ có thể thả từ dưới lên:

$$f(1, j) = j \quad (j \geq 0).$$

Xét chuyển trạng thái. Giả sử ta thả một quả trứng từ tầng w . Có hai kết quả.

Nếu trứng vỡ, ta biết $E < w$. Khi đó chỉ còn $i - 1$ quả trứng để xác định E trong $w - 1$ tầng bên dưới. Đây là bài toán con có đáp án $f(i - 1, w - 1)$, nên tổng số lần thả là $f(i - 1, w - 1) + 1$.

Nếu trứng không vỡ, ta biết $E \geq w$. Khi đó vẫn còn i quả trứng để xác định E trong $j - w$ tầng bên trên. Thí nghiệm trên $j - w$ tầng bên trên hoàn toàn tương đương với thí nghiệm trên các tầng $1, 2, \dots, j - w$, vì phương pháp và các bước không đổi, chỉ khác vị trí tuyệt đối của các tầng. Do đó số lần thả là $f(i, j - w) + 1$.

Sơ đồ chia tầng ứng với quyết định w có thể viết như sau:

$$\frac{\frac{j - w \text{ tầng bên trên}}{\text{tầng } w}}{w - 1 \text{ tầng bên dưới}}$$

Đề yêu cầu tối ưu trong trường hợp xấu nhất, nên với một quyết định w ta lấy giá trị lớn hơn trong hai nhánh, rồi lấy nhỏ nhất trên mọi quyết định:

$$f(i, j) = \min_{1 \leq w \leq j} (\max\{f(i - 1, w - 1), f(i, j - w)\} + 1). \quad (1)$$

Số trứng cần xét chắc chắn không vượt quá N , vì khi $M > N$ có thể đặt $M = N$ mà không ảnh hưởng đáp án. Vì vậy thuật toán này có độ phức tạp thời gian

$$O(MN^2) = O(N^3),$$

và độ phức tạp bộ nhớ $O(N)$ nếu dùng mảng cuộn.

3.2 Thuật toán 2

Độ phức tạp của thuật toán trên quá cao. Có thể tối ưu được không? Câu trả lời là có.

Trước hết, bài này rất giống tìm kiếm nhị phân: mỗi lần thử trúng cho ta thông tin để quyết định khoảng xử lý tiếp theo, chỉ khác là số lần trúng vỡ bị giới hạn. Giả sử số trúng không bị giới hạn. Theo lý thuyết cây quyết định, E chỉ có thể nhận một trong $n + 1$ giá trị $0, 1, 2, \dots, n$, nên cây phải có $n + 1$ lá. Do đó chiều cao cây ít nhất là

$$\lceil \log_2(n + 1) \rceil + 1,$$

tức số phép thử trong trường hợp xấu nhất cần ít nhất

$$\lceil \log_2(n + 1) \rceil$$

lần. Mặt khác, trong n số đã sắp xếp, tìm kiếm nhị phân cần $\lceil \log_2(n + 1) \rceil$ phép so sánh trong trường hợp xấu nhất; trong bài này, trường hợp không tìm thấy có thể xem là $E = 0$. Hai nhận xét này cho thấy $\lceil \log_2(n + 1) \rceil$ vừa là cận dưới, vừa là cận có thể đạt được.

Nói cách khác, với n cố định, đáp án với mọi M đều không nhỏ hơn $\lceil \log_2(n + 1) \rceil$. Một khi

$$M \geq \lceil \log_2(n + 1) \rceil,$$

bài toán trở thành bài tìm số phép so sánh xấu nhất của tìm kiếm nhị phân, và có thể trực tiếp xuất $\lceil \log_2(n + 1) \rceil$.

Nhờ vậy, độ phức tạp thời gian giảm ngay xuống

$$O(N^2 \log_2 N).$$

Điều này cho thấy tối ưu hóa quy hoạch động là rất cần thiết. Thuật toán 2 chỉ cần khảo sát tính chất của chính bài toán đã giảm được tổng số trạng thái, từ đó hạ độ phức tạp của thuật toán 1 và cải thiện đáng kể hiệu quả.

3.3 Thuật toán 3

Việc tối ưu hóa vẫn chưa dừng lại. Thực nghiệm cho thấy không thể tiếp tục giới hạn kích thước M tốt hơn nữa, nên ta xét chính phương trình quy hoạch động để tìm cách tối ưu mới.

Trong quá trình tính, ta nhận thấy

$$f(i, j) \geq f(i, j - 1) \quad (j \geq 1)$$

luôn đúng. Tính chất này có thể chứng minh bằng quy nạp.

Khi $i = 1$, vì $f(1, j) = j$ với $j \geq 0$, mệnh đề hiển nhiên đúng.

Khi $i \geq 2$, ta có

$$f(i, 0) = 0, \quad f(i, 1) = \max\{f(i - 1, 0), f(i, 0)\} + 1 = 1,$$

nên mệnh đề đúng với $j = 1$.

Giả sử với một $k > 1$, ta đã có $f(i, t) \geq f(i, t - 1)$ cho mọi $1 \leq t \leq k - 1$. Xét $j = k$:

$$f(i, k) = \min_{1 \leq w \leq k} (\max\{f(i - 1, w - 1), f(i, k - w)\} + 1),$$

$$f(i, k - 1) = \min_{1 \leq w \leq k - 1} (\max\{f(i - 1, w - 1), f(i, k - 1 - w)\} + 1).$$

Với $1 \leq w \leq k-1$, vì $k-1-w < k-w \leq k-1$, theo giả thiết quy nạp:

$$f(i, k-w) \geq f(i, k-1-w).$$

Do đó

$$\max\{f(i-1, w-1), f(i, k-w)\} + 1 \geq \max\{f(i-1, w-1), f(i, k-1-w)\} + 1.$$

Đặt

$$t = \min_{1 \leq w \leq k-1} (\max\{f(i-1, w-1), f(i, k-w)\} + 1),$$

suy ra

$$t \geq f(i, k-1). \quad (2)$$

Mặt khác,

$$f(i, k) = \min\{t, \max\{f(i-1, k-1), f(i, 0)\} + 1\} = \min\{t, f(i-1, k-1) + 1\}.$$

Trong công thức của $f(i, k-1)$, lấy $w = k-1$, ta được

$$f(i, k-1) \leq \max\{f(i-1, k-2), f(i, 0)\} + 1 = f(i-1, k-2) + 1 \leq f(i-1, k-1) + 1.$$

Cùng với (2), cả hai phần tử trong phép min đều không nhỏ hơn $f(i, k-1)$, nên

$$f(i, k) \geq f(i, k-1).$$

Tính đơn điệu được chứng minh:

$$f(i, j) \geq f(i, j-1) \quad (j \geq 1). \quad (3)$$

Từ kết luận này, ta có thể dùng tìm kiếm nhị phân trong chuyển trạng thái.

Nếu

$$f(i-1, w-1) < f(i, j-w),$$

thì với mọi $w' < w$,

$$f(i, j-w') \geq f(i, j-w).$$

Vì vậy

$$\max\{f(i-1, w'-1), f(i, j-w')\} + 1 \geq f(i, j-w') + 1 \geq f(i, j-w) + 1 = \max\{f(i-1, w-1), f(i, j-w)\} + 1.$$

Quyết định w' không thể tốt hơn w .

Ngược lại, nếu

$$f(i-1, w-1) \geq f(i, j-w),$$

thì với mọi $w' > w$,

$$f(i-1, w'-1) \geq f(i-1, w-1),$$

nên

$$\begin{aligned} & \max\{f(i-1, w'-1), f(i, j-w')\} + 1 \\ & \geq f(i-1, w'-1) + 1 \\ & \geq f(i-1, w-1) + 1 \\ & = \max\{f(i-1, w-1), f(i, j-w)\} + 1. \end{aligned}$$

Quyết định w' cũng không thể tốt hơn w .

Có thể hình dung hai hàm

$$f(i-1, w-1) \quad \text{và} \quad f(i, j-w)$$

theo biến w : hàm thứ nhất không giảm, hàm thứ hai không tăng. Đồ thị của

$$\max\{f(i-1, w-1), f(i, j-w)\} + 1$$

đạt cực tiểu ở gần điểm hai đồ thị giao nhau. Vì vậy trong khi tìm quyết định tốt nhất w_{best} , mỗi lần so sánh hai về tại trung điểm ta có thể bỏ đi một nửa khoảng quyết định. Sau không quá $\lceil \log_2(n+1) \rceil$ bước, ta tìm được quyết định tốt nhất.

Như vậy, nhờ tính đơn điệu của $f(i, j)$, chi phí chuyển trạng thái giảm xuống $O(\log_2 N)$, và độ phức tạp thời gian của thuật toán còn

$$O(N \log_2^2 N).$$

Mức này đã xử lý được trường hợp N khá lớn.

Từ thuật toán 2 đến thuật toán 3, ta đã dùng tính đơn điệu của hàm quy hoạch động $f(i, j)$ để giảm độ phức tạp của phần chuyển trạng thái. Điều này cho thấy ngay trong bản thân thuật toán vẫn có không gian tối ưu, tạo tiền đề để tìm thuật toán bậc thấp hơn.

3.4 Thuật toán 4

Sau khi nghiên cứu thuật toán 3, ta nảy sinh một suy nghĩ: vì hiện nay vẫn phải tính mọi $f(i, j)$, muốn có thuật toán hiệu quả hơn thì chỉ còn có thể động vào chuyển trạng thái. Bước này đang là $O(\log_2 N)$, vẫn chưa đủ lý tưởng.

Để tối ưu tiếp, cần đào sâu hơn vào phương trình:

$$f(i, j) = \min_{1 \leq w \leq j} (\max\{f(i-1, w-1), f(i, j-w)\} + 1).$$

Rõ ràng, với mọi $1 \leq w \leq j$:

$$f(i, j) \leq \max\{f(i-1, w-1), f(i, j-w)\} + 1.$$

Lấy $w = 1$:

$$f(i, j) \leq \max\{f(i-1, 0), f(i, j-1)\} + 1 = f(i, j-1) + 1.$$

Tức là

$$f(i, j) \leq f(i, j-1) + 1 \quad (j \geq 1). \quad (4)$$

Kết hợp với (3):

$$f(i, j-1) \leq f(i, j) \leq f(i, j-1) + 1 \quad (j \geq 1). \quad (5)$$

Đây là một kết luận rất quan trọng. Từ đó suy ra: nếu tồn tại một quyết định w làm được $f(i, j) = f(i, j-1)$, thì $f(i, j) = f(i, j-1)$; nếu mọi quyết định w đều không làm được điều này, thì $f(i, j) = f(i, j-1) + 1$, và chắc chắn tồn tại quyết định đạt giá trị ấy.

Ta đặt một con trỏ p sao cho luôn thỏa

$$f(i, p) < f(i, j-1), \quad f(i, p+1) = f(i, j-1).$$

Rõ ràng khi đó

$$\begin{aligned} f(i, p) &= f(i, j-1) - 1, \\ f(i, p+1) &= f(i, p+2) = \dots = f(i, j-1). \end{aligned} \quad (6)$$

Khi tính $f(i, j)$, đặt $p = j - w$, tức $w = j - p$. Khi đó

$$tmp = \max\{f(i-1, w-1), f(i, j-w)\} + 1 = \max\{f(i-1, j-p-1), f(i, p)\} + 1.$$

Nếu

$$f(i, p) \geq f(i-1, j-p-1),$$

thì

$$tmp = f(i, p) + 1 = f(i, j-1) - 1 + 1 = f(i, j-1).$$

Quyết định hiện tại làm được $f(i, j) = f(i, j-1)$, nên $f(i, j) = f(i, j-1)$.

Nếu

$$f(i, p) < f(i-1, j-p-1),$$

ta xét mọi vị trí p' khác.

Với $p' < p$, do tính đơn điệu:

$$f(i-1, j-p'-1) \geq f(i-1, j-p-1) > f(i, p).$$

Suy ra

$$\max\{f(i, p'), f(i-1, j-p'-1)\} + 1 \geq f(i-1, j-p'-1) + 1 \geq f(i-1, j-p-1) + 1 > f(i, p) + 1 = f(i, j-1).$$

Giá trị là số nguyên, nên nó ít nhất là $f(i, j-1) + 1$ và không thể tạo ra $f(i, j) = f(i, j-1)$.

Với $p' = p$, ta cũng có

$$\max\{f(i, p'), f(i-1, j-p'-1)\} + 1 \geq f(i-1, j-p'-1) + 1 > f(i, p') + 1 = f(i, j-1),$$

nên vẫn không thể tạo ra $f(i, j) = f(i, j-1)$.

Với $p' > p$, theo (6) có $f(i, p') = f(i, j-1)$, do đó

$$\max\{f(i, p'), f(i-1, j-p'-1)\} + 1 \geq f(i, p') + 1 = f(i, j-1) + 1.$$

Trường hợp này cũng không thể tạo ra $f(i, j) = f(i, j-1)$.

Tóm lại, khi $f(i, p) < f(i-1, j-p-1)$, không có quyết định nào làm được $f(i, j) = f(i, j-1)$, vì vậy

$$f(i, j) = f(i, j-1) + 1.$$

Do đó chỉ cần so sánh $f(i, p)$ với $f(i-1, j-p-1)$ là có thể trực tiếp xác định giá trị $f(i, j)$. Chuyển trạng thái giảm xuống $O(1)$, kéo theo độ phức tạp thời gian của thuật toán giảm còn

$$O(N \log_2 N).$$

Khi cài đặt cần chú ý hai điểm. Thứ nhất, $f(i, 1)$ phải xử lý riêng bằng cách gán $f(i, 1) = 1$, vì giá trị ban đầu $p = 0$ không thỏa $f(i, p) = f(i, j-1) - 1$. Thứ hai, nếu $f(i, j) = f(i, j-1) + 1$ thì cập nhật $p \leftarrow j - 1$.

Từ thuật toán 3 đến thuật toán 4, ta xuất phát từ phần chưa tối ưu hoàn toàn, tức là chuyển trạng thái, rồi tiếp tục khai thác đặc tính của phương trình quy hoạch động. Nhờ tìm đúng điểm chia p dùng để tính $f(i, j)$, chi phí chuyển trạng thái trở thành $O(1)$, và hiệu quả thuật toán được nâng thêm một bước.

3.5 Tóm lược

Từ thuật toán 1 đến thuật toán 4, ta đã thực hiện ba bước tối ưu hóa.

Thuật toán 1 xây dựng mô hình quy hoạch động, là nền tảng cho các thuật toán tối ưu phía sau.

Thuật toán 2 giới hạn miền giá trị của M trong $\lceil \log_2(n+1) \rceil$, qua đó tối ưu từ phía tổng số trạng thái.

Thuật toán 3 dùng tính đơn điệu của $f(i, j)$ để cải tiến phần chuyển trạng thái.

Thuật toán 4 khai thác một tính chất đặc biệt khác của $f(i, j)$, làm cho thời gian chuyển trạng thái trở thành $O(1)$, tức là tiếp tục cải tiến phần còn chưa vừa ý trong thuật toán cũ.

Đến đây, mô hình quy hoạch động đã qua nhiều lần tối ưu dường như không còn chỗ cải tiến. Nhưng suy nghĩ thêm, ta lại tìm được một mô hình quy hoạch động khác. Với mô hình mới, thuật toán 5 có thể giảm độ phức tạp xuống $O(\sqrt{N})$.

3.6 Thuật toán 5

Ta định nghĩa một hàm quy hoạch động mới $g(i, j)$: dùng j quả trứng và thử i lần, trong trường hợp xấu nhất có thể xác định E nhiều nhất trên bao nhiêu tầng.

Rõ ràng, dù có bao nhiêu quả trứng, nếu chỉ thử một lần thì chỉ xác định được một tầng:

$$g(1, j) = 1 \quad (j \geq 1).$$

Nếu chỉ có một quả trứng và được thử i lần, trong trường hợp xấu nhất ta xác định được E trên i tầng:

$$g(i, 1) = i \quad (i \geq 1).$$

Chuyển trạng thái cũng rất đơn giản. Giả sử lần đầu ta thả một quả trứng từ một tầng nào đó. Nếu trứng vỡ, trong $i-1$ lần còn lại ta phải dùng $j-1$ quả trứng để xác định E ở các tầng bên dưới. Để $g(i, j)$ lớn nhất, số tầng bên dưới cũng phải lớn nhất, và đây là bài toán con $g(i-1, j-1)$. Nếu trứng không vỡ, trong $i-1$ lần còn lại ta dùng j quả trứng để xác định E ở các tầng bên trên, được $g(i-1, j)$ tầng.

Sơ đồ tầng ứng với lần thả đầu tiên:

$$\frac{\frac{g(i-1, j) \text{ tầng bên trên}}{\text{tầng thả đầu tiên}}}{g(i-1, j-1) \text{ tầng bên dưới}} \quad \text{tổng cộng } g(i-1, j-1) + g(i-1, j) + 1 \text{ tầng.}$$

Vì vậy:

$$g(i, j) = g(i-1, j-1) + g(i-1, j) + 1. \quad (7)$$

Mục tiêu là tìm một x thỏa

$$g(x-1, M) < N, \quad g(x, M) \geq N. \quad (8)$$

Đáp án chính là x ; trường hợp $x=1$ cần xét riêng.

Thoạt nhìn thuật toán này là $O(N \log_2 N)$: biến i là số lần thử, tối đa N ; biến j là số trứng, tối đa M , tức khoảng $\log_2 N$ sau giới hạn ở thuật toán 2; chuyển trạng thái là $O(1)$. Nhưng thực tế không phải như vậy. Ta chứng minh điều này dưới đây.

Quan sát nhanh cho thấy hàm $g(i, j)$ rất giống hệ số tổ hợp $\binom{i}{j}$:

$g(i, j)$	$\binom{i}{j}$
$g(1, j) = 1 \ (j \geq 1)$	$\binom{1}{1} = 1, \binom{1}{j} = 0 \ (j \geq 2)$
$g(i, 1) = i \ (i \geq 1)$	$\binom{i}{1} = i \ (i \geq 1)$
$g(i, j) = g(i-1, j-1) + g(i-1, j) + 1$	$\binom{i}{j} = \binom{i-1}{j-1} + \binom{i-1}{j}$

Dựa trên điều kiện biên và công thức truy hồi, ta dễ dàng chứng minh bằng quy nạp rằng với mọi $i, j \geq 1$:

$$g(i, j) \geq \binom{i}{j}.$$

Từ (8) suy ra:

$$\binom{x-1}{M} \leq g(x-1, M) < N,$$

tức là

$$\binom{x-1}{M} < N, \quad \frac{(x-1)(x-2) \cdots (x-M)}{M!} < N. \quad (9)$$

Ta dùng một bổ đề đơn giản: khi $1 \leq N \leq 3$ hoặc $N \geq 17$,

$$(\log_2 N)^2 < N.$$

Bổ đề này có thể chứng minh bằng đồ thị hàm số. Với $4 \leq N \leq 16$, $(\log_2 N)^2 \geq N$, nhưng độ lệch rất nhỏ và không ảnh hưởng đến phân tích tiệm cận.

Nếu $x < M$, thì

$$xM < M^2 \leq (\log_2 N)^2 < N.$$

Nếu $x \geq M$, từ (9):

$$(x-M)^M \leq (x-1)(x-2) \cdots (x-M) < N \cdot M!.$$

Do đó

$$x-M \leq \sqrt[M]{N \cdot M!} \leq \sqrt[M]{N \cdot M^M} = M \sqrt[M]{N},$$

và

$$x \leq M + M \sqrt[M]{N} = M \left(1 + \sqrt[M]{N}\right).$$

Suy ra

$$xM \leq M^2 \left(1 + \sqrt[M]{N}\right). \quad (10)$$

Mặt khác, vì $\sqrt[M-1]{N}$ xấp xỉ N ở thang đang xét và $N > (\log_2 N)^2 \geq M^2$, ta có

$$\sqrt[M-1]{N} \geq M^2,$$

tính chất này vẫn đúng với $M = 1$ khi xét riêng. Từ đó có thể biến đổi:

$$\frac{M-1}{M} \log_2 N \geq \log_2 M^2,$$

$$\frac{1}{M} \log_2 N + \log_2 M^2 \leq \log_2 N,$$

$$\log_2 \sqrt[M]{N} + \log_2 M^2 \leq \log_2 N.$$

Vì $\sqrt[M]{N}$ gần với $\sqrt[M]{N} + 1$ về bậc tăng trưởng, ta thu được

$$\log_2 \left((\sqrt[M]{N} + 1) M^2 \right) \leq \log_2 N,$$

nên

$$M^2 \left(1 + \sqrt[M]{N}\right) \leq N.$$

Kết hợp với (10):

$$xM \leq N.$$

Điều này cho thấy lượng tính toán thực tế của hàm $g(i, j)$ là $O(N)$. Vậy vì sao nó có thể thành $O(\sqrt{N})$?

Từ

$$\frac{(x-1)(x-2)\cdots(x-M)}{M!} < N,$$

ta có

$$(x-1)(x-2)\cdots(x-M) < N \cdot M!.$$

Điều này cho thấy khi M không quá lớn, x có cùng bậc với $\sqrt[M]{N}$. Trong thực tế, chỉ khi $M = 1$ thì $x = N$ và $xM = N$; ngay khi $M > 1$, xM giảm xuống bậc $O(\sqrt{N})$. Vì vậy chỉ cần xử lý riêng trường hợp $M = 1$ là có thể đưa độ phức tạp thời gian xuống

$$O(\sqrt{N}),$$

và dùng mảng cuộn để đưa độ phức tạp bộ nhớ xuống

$$O(M) = O(\log_2 N).$$

Trong mô hình quy hoạch động mới, ta tìm được một phương pháp tốt hơn hẳn các thuật toán trước. Điều này nhắc ta không nên luôn bị trói trong lối nghĩ cũ. Đổi góc nhìn thường tạo ra hiệu quả bất ngờ. Nếu chỉ hài lòng với thuật toán 4, ta sẽ không mở rộng được tư duy để tìm lời giải hiệu quả hơn. Vì vậy, nhìn bài toán từ nhiều góc độ cũng là một yếu tố rất quan trọng khi tối ưu hóa quy hoạch động.

4 Tổng kết

Bài viết đã bàn về năm thuật toán có hiệu năng khác nhau cho bài *Trúng đại bàng*. Bảng sau so sánh các thuật toán ấy:

Thuật toán	Thời gian	Bộ nhớ	Phương pháp tối ưu
1	$O(N^3)$	$O(N)$	Xây dựng mô hình quy hoạch động ban đầu.
2	$O(N^2 \log_2 N)$	$O(N)$	Khảo sát tính chất của bài toán để giảm tổng số trạng thái.
3	$O(N \log_2^2 N)$	$O(N)$	Nghiên cứu phương trình quy hoạch động, tìm đặc tính của nó và tối ưu phần chuyển trạng thái.
4	$O(N \log_2 N)$	$O(N)$	Đào sâu thêm đặc tính của phương trình quy hoạch động để tiếp tục giảm chi phí chuyển trạng thái.
5	$O(\sqrt{N})$	$O(\log_2 N)$	Xây dựng mô hình quy hoạch động mới và xét lại bài toán từ một góc nhìn khác.

Từ bảng này, có thể thấy rất rõ rằng tối ưu hóa có thể cải thiện đáng kể hiệu quả của thuật toán quy hoạch động. Phương pháp tối ưu hóa quy hoạch động cũng rất đa dạng. Điều này đòi hỏi khi nghiên cứu bài toán, ta phải phân tích sâu, dám đổi mới, không tự bằng lòng và liên tục cải tiến; chỉ như vậy tối ưu hóa mới thật sự đi vào thực chất.

Trong các bài toán thực tế, dù kỹ thuật tối ưu có muôn hình vạn trạng, tư tưởng cốt lõi vẫn không đổi: tìm phần còn chưa hoàn hảo của thuật toán quy hoạch động để cải tiến thêm, hoặc mở một con đường khác bằng cách xây dựng mô hình mới hiệu quả hơn. Trong quá trình tối ưu cụ thể, ta thường bắt đầu từ việc giảm số trạng thái, khai thác đặc tính của phương trình quy hoạch động, giảm độ phức tạp của chuyển trạng thái, và xây dựng mô hình mới. Đó chính là phương pháp chung để tối ưu hóa thuật toán quy hoạch động.

Dĩ nhiên, mọi sự vật đều có cả tính phổ biến lẫn tính đặc thù. Khi phương pháp chung khó giải quyết vấn đề, dùng phương pháp đặc thù như bất đẳng thức tứ giác hoặc tối ưu bằng độ dốc cũng là lựa chọn tốt. Vì vậy, chỉ khi nắm vững linh hoạt cả phương pháp chung lẫn phương pháp đặc thù, ta mới thật sự giải quyết hiệu quả các vấn đề tối ưu hóa quy hoạch động.

5 Lời kết

Bài viết này chỉ bàn về tư tưởng bản chất và phương pháp chung khi tối ưu hóa thuật toán quy hoạch động. Trên thực tế, tư tưởng tối ưu cực kỳ quan trọng và có mặt ở khắp nơi. Tối ưu hóa có thể biến một thuật toán vốn kém hiệu quả thành một thuật toán rất tốt, làm quy mô bài toán xử lý được tăng lên hàng chục, hàng trăm, thậm chí hàng nghìn lần. Quan trọng hơn, tư tưởng tối ưu có phạm vi ứng dụng rất rộng. Dù trong thi tin học hay trong sản xuất và đời sống hằng ngày, tư tưởng tối ưu đều giữ vai trò quan trọng. Khi có tư tưởng tối ưu, ta sẽ không thỏa mãn với phương pháp hiện tại, mà sẽ liên tục khai phá, đổi mới và tạo ra phương pháp tốt hơn.

Tối ưu, rồi tối ưu nữa, là để thuật toán tiếp tục hoàn thiện. Đồng thời, quá trình ấy mở rộng tư duy, rèn luyện năng lực phân tích sâu vấn đề, bồi dưỡng tinh thần không ngừng tiến bộ, và cũng rất có ích cho nghiên cứu khoa học sau này.

Bài viết chỉ phân tích sơ bộ một bài thi tin học, mong có thể gợi thêm suy nghĩ, giúp chúng ta hiểu rõ hơn vai trò quan trọng của tư tưởng tối ưu, biết dùng tốt và linh hoạt tư tưởng ấy để giải quyết vấn đề tốt hơn, hiệu quả hơn.

Tài liệu tham khảo

1. Yan Weimin, Wu Weimin, *Cấu trúc dữ liệu*, tái bản lần 2, NXB Đại học Thanh Hoa, Bắc Kinh, 1992.
2. Wu Wenhui, Wang Jiande, *Hướng dẫn thi Olympic Tin học: thuật toán và lập trình tổ hợp toán học bằng Pascal*, NXB Đại học Thanh Hoa, Bắc Kinh, 1997.
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, Second Edition, The MIT Press, 2001.
4. Ural Online Judge, acm.timus.ru. Bài gốc: <http://acm.timus.ru/problem.aspx?space=1&num=1223>. Để tiện thảo luận, bài viết này có điều chỉnh một số phần của đề.

A Đề gốc được thảo luận

Chernobyl's Eagle on a Roof

Giới hạn thời gian: 1.0 giây. Giới hạn bộ nhớ: 1000 KB.

Ngày xưa ngày xưa, một con đại bàng làm tổ trên mái một tòa nhà rất cao. Thời gian trôi qua và trong tổ xuất hiện một số quả trứng. Vào một ngày nắng đẹp, Niels Bohr đi dạo trên mái nhà. Ông bỗng nói:

“Ồ! Mọi quả trứng chắc chắn có cùng độ bền, nên tồn tại một số không âm E sao cho nếu thả trứng từ tầng số E thì nó không vỡ, và điều này cũng đúng với mọi tầng thấp hơn E ; nhưng nếu thả từ tầng $E + 1$ thì trứng sẽ vỡ, và điều này cũng đúng với mọi tầng cao hơn E . Bây giờ giáo sư Bohr sẽ tổ chức một loạt thí nghiệm, tức là các lần thả. Mục tiêu của thí nghiệm là xác định hằng số E . Rõ ràng có thể tìm E bằng cách thả trứng tuần tự từng tầng từ tầng thấp nhất. Nhưng cũng có những chiến lược khác bảo đảm tìm

được E với số thí nghiệm ít hơn nhiều. Hãy tìm số lần thả trứng ít nhất đủ để chắc chắn xác định được E , ngay cả trong trường hợp xấu nhất. Chú ý rằng các quả trứng đã thả mà không vỡ có thể được dùng lại trong các thí nghiệm sau.”

Số tầng là một số nguyên dương, còn E là một số không âm. Cả hai đều không vượt quá 1000. Các tầng được đánh số bằng các số nguyên dương bắt đầu từ 1. Nếu một quả trứng không vỡ dù được thả từ tầng cao nhất, ta có thể xem rằng E cũng đã được xác định và bằng tổng số tầng.

Dữ liệu vào. Dữ liệu gồm nhiều bộ kiểm thử. Mỗi dòng là một bộ kiểm thử, gồm hai số cách nhau bởi một dấu cách: trước hết là số trứng, sau đó là số tầng. Dữ liệu kết thúc bằng dòng chứa một số 0.

Dữ liệu ra. Với mỗi bộ kiểm thử, in trên một dòng số thí nghiệm ít nhất mà Niels Bohr phải thực hiện trong trường hợp xấu nhất.

Ví dụ.

Input	Output
1 10	10
2 5	3
0	

B Các chương trình liên quan

Chương trình eagle_1.cpp ứng với thuật toán 1 có độ phức tạp quá cao nên bị quá thời gian trên Ural Online Judge. Các chương trình eagle_2.cpp, eagle_3.cpp, eagle_4.cpp và eagle_5.cpp đều được kiểm thử đạt trên Ural Online Judge.

eagle_1.cpp

```
#include<iostream.h>
#define maxn 1100
#define maxnum 10000000000

long n,eggnum,now,old,f[2][maxn+1];

void init()
{
    long i;
    now=1;
    f[now][0]=0;
    for (i=1; i<=n; i++)
        f[now][i]=i;
}

long max(long a,long b)
{
    if (a>b)
        return(a);
    else return(b);
}
```

```

}

void work()
{
long i,j,w,temp;
for (i=2; i<=eggnum; i++)
{
old=now;
now=1-now;
f[now][0]=0;
for (j=1; j<=n; j++)
{
f[now][j]=maxnum;
for (w=1; w<=j; w++)
{
temp=max(f[old][w-1],f[now][j-w])+1;
if (temp<f[now][j])
f[now][j]=temp;
}
}
}
}

void output()
{
cout<<f[now][n]<<endl;
}

int main()
{
while (1)
{
cin>>eggnum;
if (eggnum==0)
break;
else cin>>n;
init();
work();
output();
}
return 0;
}

```

eagle_2.cpp

```

#include<iostream.h>
#include<math.h>
#define maxn 1100

```

```

#define maxnum 1000000000

long n,eggnum,now,old,f[2][maxn+1];

void init()
{
    long i;
    now=1;
    f[now][0]=0;
    for (i=1; i<=n; i++)
        f[now][i]=i;
}

long max(long a,long b)
{
    if (a>b)
        return(a);
    else return(b);
}

void work()
{
    long i,j,w,temp;
    for (i=2; i<=eggnum; i++)
    {
        old=now;
        now=1-now;
        f[now][0]=0;
        for (j=1; j<=n; j++)
        {
            f[now][j]=maxnum;
            for (w=1; w<=j; w++)
            {
                temp=max(f[old][w-1],f[now][j-w])+1;
                if (temp<f[now][j])
                    f[now][j]=temp;
            }
        }
    }
}

void output()
{
    cout<<f[now][n]<<endl;
}

int main()
{

```

```

long temp;
while (1)
{
    cin>>eggnum;
    if (eggnum==0)
        break;
    else cin>>n;
    temp=long (floor(log(n+0.0)/log(2.0))+1.0);
    if (eggnum>=temp)
        cout<<temp<<endl;
    else {
        init();
        work();
        output();
    }
}
return 0;
}

```

eagle_3.cpp

```

#include<iostream.h>
#include<math.h>
#define maxn 1100
#define maxnum 1000000000

long n,eggnum,now,old,f[2][maxn+1];

void init()
{
    long i;
    now=1;
    f[now][0]=0;
    for (i=1; i<=n; i++)
        f[now][i]=i;
}

long max(long a,long b)
{
    if (a>b)
        return(a);
    else return(b);
}

void calc(long i,long j)
{
    long w,temp,start,stop,mid;
    f[now][j]=maxnum;

```

```

start=1; stop=j;
while (start<=stop)
{
    mid=(start+stop)/2;
    if (f[old][mid-1]>f[now][j-mid])
    {
        if (f[old][mid-1]+1<f[now][j])
            f[now][j]=f[old][mid-1]+1;
        stop=mid-1;
    }
    else if (f[old][mid-1]<f[now][j-mid])
    {
        if (f[now][j-mid]+1<f[now][j])
            f[now][j]=f[now][j-mid]+1;
        start=mid+1;
    }
    else {
        f[now][j]=f[now][j-mid]+1;
        return;
    }
}
}

void work()
{
    long i,j;
    for (i=2; i<=eggnum; i++)
    {
        old=now;
        now=1-now;
        f[now][0]=0;
        for (j=1; j<=n; j++)
            calc(i,j);
    }
}

void output()
{
    cout<<f[now][n]<<endl;
}

int main()
{
    long temp;
    while (1)
    {
        cin>>eggnum;
        if (eggnum==0)

```

```

        break;
        else cin>>n;
temp=long (floor(log(n+0.0)/log(2.0))+1.0);
if (eggnum>=temp)
    cout<<temp<<endl;
    else {
        init();
        work();
        output();
    }
}
return 0;
}

```

eagle_4.cpp

```

#include<iostream.h>
#include<math.h>
#define maxn 1100
#define maxnum 1000000000

long n,eggnum,now,old,f[2][maxn+1];

void init()
{
    long i;
    now=1;
    f[now][0]=0;
    for (i=1; i<=n; i++)
        f[now][i]=i;
}

void work()
{
    {
    long i,j,p;
    for (i=2; i<=eggnum; i++)
        {
            old=now;
            now=1-now;
            p=f[now][0]=0;
            f[now][1]=1; // special
            for (j=2; j<=n; j++)
                if (f[now][p]>=f[old][j-p-1])
                    f[now][j]=f[now][j-1];
                else {
                    f[now][j]=f[now][j-1]+1;
                    p=j-1;
                }
        }
    }
}

```



```

    }
}

void output()
{
cout<<f[now][n]<<endl;
}

int main()
{
long temp;
while (1)
{
cin>>eggnum;
if (eggnum==0)
break;
else cin>>n;
temp=long (floor(log(n+0.0)/log(2.0))+1.0);
if (eggnum>=temp)
cout<<temp<<endl;
else {
init();
work();
output();
}
}
return 0;
}

```

eagle_5.cpp

```

#include<iostream.h>
#include<math.h>
#define maxlogn 20
#define maxnum 10000000000

long n,eggnum,now,old,g[maxlogn+1];

void init()
{
long i;
now=1;
for (i=1; i<=eggnum; i++)
g[i]=1;
}

void work()
{

```

```

long i,j,p;
for (i=2; i<=n; i++)
{
    for (j=eggnum; j>=2; j--)
    {
        g[j]=g[j-1]+g[j]+1;
        if ((j==eggnum)&&(g[j]>=n))
        {
            cout<<i<<endl;
            return;
        }
    }
    g[1]=i;
    if ((eggnum==1)&&(g[1]>=n))
    {
        cout<<i<<endl;
        return;
    }
}

int main()
{
    long temp;
    while (1)
    {
        cin>>eggnum;
        if (eggnum==0)
            break;
        else cin>>n;
        temp=long (floor(log(n+0.0)/log(2.0))+1.0);
        if (eggnum>=temp)
            cout<<temp<<endl;
        else {
            init();
            if(g[eggnum]>=n)
                cout<<1<<endl;
            else if (eggnum==1)
                cout<<n<<endl;
            else work();
        }
    }
    return 0;
}

```